

Compute4Me Study Notes — Section 2: Classical Distributed Deep Learning Architectures

Context: This document is part of a series walking through the research landscape for *Compute4Me*, a distributed deep learning compute fabric where any Ubuntu machine can join by pulling a Docker image, self-register with a master node, receive workload partitions, and return results. Section 2 covers the classical building blocks: the Parameter Server (PS) and All-reduce architectures, plus the key systems built on them.

The Core Problem These Architectures Solve

When training a deep neural network on multiple machines simultaneously, every machine needs to stay in sync on one question: **what are the current model weights?**

Each worker (GPU/node) does the same loop:

1. Pull the latest model parameters θ
2. Compute gradients ∇L on its local mini-batch
3. Somehow share and merge those gradients so all workers update one consistent model

Steps 1 and 2 are easy. Step 3 is the hard distributed systems problem. The Parameter Server and All-reduce are the two dominant answers.

2.1 Parameter Server (PS) Architecture

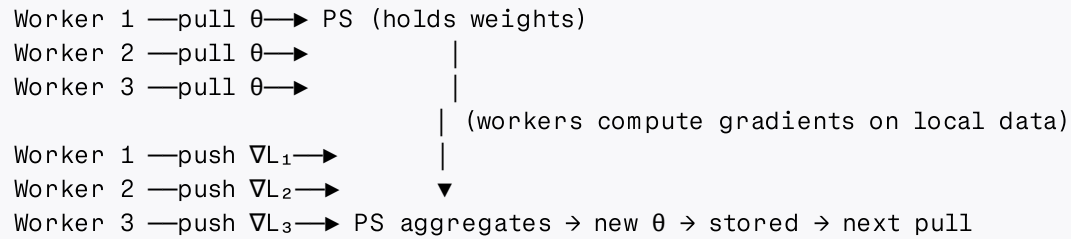
What It Is

A Parameter Server is a dedicated group of processes (the "servers") that acts as a centralized, distributed key-value store for model parameters. Worker processes (the "workers") interact with it via two operations:

- **pull:** "Give me the current weights for layer X."
- **push:** "Here are the gradients I computed; please apply them."

The PS aggregates incoming gradients from all workers, applies an optimizer step (e.g., SGD update), and stores the new weights, which workers can pull again on the next iteration.

How It Works (Step by Step)



Synchronous vs Asynchronous

- **Synchronous PS:** The PS waits for all N workers to push gradients before applying the update. This keeps every worker perfectly in sync but creates a "straggler" problem: the whole job blocks waiting for the slowest machine.
- **Asynchronous PS (Downpour SGD):** The PS applies gradients from each worker as they arrive, without waiting for others. Workers may be operating on slightly stale parameters. Faster in practice for large clusters, but convergence is noisier.

For Compute4Me — where nodes have different speeds — **asynchronous PS is more practical**. You don't want one RTX 3070 worker blocked waiting for a slow VM to finish.

Key System: DistBelief (Google, 2012)

DistBelief was Google's internal framework for training large neural networks using thousands of CPU cores via an asynchronous PS architecture.

- Trained networks with billions of parameters at Google scale.
- Demonstrated that asynchronous Downpour SGD can scale to very large clusters while tolerating slow or failed workers.
- Introduced the concept of **model sharding**: different PS nodes hold different slices of parameters, so no single server is a memory bottleneck.

Paper: "Large Scale Distributed Deep Networks" (Dean et al., NIPS 2012)

Link: <https://research.google/pubs/large-scale-distributed-deep-networks/>

What you can take from this for Compute4Me:

- The PS idea maps directly: your master node (or a dedicated PS service running on it) holds global model state, and worker containers push/pull updates.
- Asynchronous updates mean you can tolerate nodes of different speeds without stalling the whole job.

Difficulty to use: Not directly usable as an off-the-shelf library; DistBelief was Google-internal and was replaced by TensorFlow. However, the *idea* is standard and can be re-implemented relatively simply using Python + Redis or gRPC.

Difficulty to reimplement from scratch:

- Simple async PS: moderate. A few hundred lines of Python with gRPC or ZeroMQ.
- Production-grade PS (like DistBelief, handling millions of parameters, fault tolerance, efficient sharding): very hard.

Key System: GeePS (CMU, EuroSys 2016)

GeePS extended the PS idea specifically for GPU clusters. The key insight: storing parameters in GPU memory and designing the PS logic around the GPU memory hierarchy dramatically reduces the overhead of pulling/pushing parameters.

- A small cluster of GPU machines with GeePS outperformed a 108-node CPU cluster on deep learning throughput.
- Achieved up to 13× speedup compared to an optimized single-node baseline using 16 GPU machines.
- Optimized data placement (keeping "hot" parameters in GPU memory, overflowing "cold" ones to CPU/disk) automatically.

Paper: "GeePS: Scalable deep learning on distributed GPUs with a GPU-specialized parameter server" (Cui et al., EuroSys 2016)

PDF: <https://www.pdl.cmu.edu/PDL-FTP/CloudComputing/GeePS-cui-eurosys16.pdf>

ACM entry: <https://dl.acm.org/doi/10.1145/2901318.2901323>

What you can take from this for Compute4Me:

- Highlights that naive PS designs waste time on CPU ↔ GPU data movement; a GPU-aware PS stores weights where the compute happens.
- For Compute4Me, this is relevant when designing how worker containers store and update intermediate model state: ideally keep parameters in GPU memory, not passing through CPU on every gradient exchange.

Difficulty to use: GeePS is a research prototype (C++ codebase). Not practically usable off-the-shelf.

Difficulty to reimplement: Very high. Requires deep understanding of CUDA memory management, PCIe topology, and custom CUDA kernels for efficient data movement.

Practical alternative: Modern PyTorch + NCCL already handles most of this automatically. You don't need to build GeePS-style logic yourself; PyTorch DDP and NCCL do the GPU-memory-aware communication under the hood.

2.2 All-reduce Architecture

What It Is

Instead of routing gradients through a central PS, all-reduce lets workers **talk directly to each other** in a coordinated pattern to compute the average of their gradients. After all-reduce completes, every worker holds the same averaged gradient $\bar{g} = \frac{1}{N} \sum_{i=1}^N g_i$, and they all perform the same optimizer step independently, staying in sync without any central server.

How Ring All-reduce Works

The most common pattern is **ring all-reduce**, which is efficient because each node sends and receives from exactly two neighbours:

```
Worker 0 — sends chunk → Worker 1
Worker 1 — sends chunk → Worker 2
Worker 2 — sends chunk → Worker 3
Worker 3 — sends chunk → Worker 0
```

After two phases (reduce-scatter + all-gather), every worker has the full averaged gradient. Total data transferred per worker: $2 \cdot \frac{N-1}{N} \cdot |\text{gradient}|$, which approaches $2 \cdot |\text{gradient}|$ for large N — nearly bandwidth-optimal regardless of cluster size.

Synchronous Only

All-reduce is inherently synchronous: all workers must participate and complete before anyone can proceed. This means stragglers stall everyone — the opposite trade-off from async PS.

- **Good for:** Homogeneous clusters with reliable nodes and fast interconnects (data centers, HPC, NVLink clusters).
- **Problematic for:** Heterogeneous volunteer clusters like Compute4Me where nodes have very different speeds.

This is an important reason why a pure all-reduce approach may not be ideal for Compute4Me — but it's still worth knowing because internal container training can still use all-reduce within a group of similar-speed nodes.

Key Library: NCCL (NVIDIA Collective Communications Library)

NCCL is NVIDIA's low-level library that implements high-performance collective operations (all-reduce, broadcast, all-gather, reduce-scatter) across multiple GPUs, whether they're on the same node or different nodes.

- Automatically detects and exploits NVLink, PCIe, InfiniBand, and TCP/IP depending on the hardware.
- Used internally by PyTorch DDP, Horovod, DeepSpeed, and most modern distributed DL frameworks.
- You almost never call NCCL directly — frameworks call it for you.

GitHub: <https://github.com/NVIDIA/ncccl>

Difficulty to use: You don't use NCCL directly; it's a backend. Just install it (usually comes with CUDA toolkit) and set `backend='nccl'` in PyTorch distributed init.

Difficulty to reimplement: Extremely high. NCCL contains hand-optimized CUDA kernels and topology-aware algorithms. Nobody reimplements this.

Key System: Horovod (Uber, 2018)

Horovod is Uber's open-source library for distributed DL that makes multi-node training as simple as a few extra lines in your existing training script. It uses ring all-reduce (via NCCL or MPI) under the hood.

The pitch: You already have a working PyTorch/TensorFlow training script. Horovod lets you scale it to N GPUs across M machines with minimal code changes:

```
import horovod.torch as hvd

hvd.init()
torch.cuda.set_device(hvd.local_rank())

# Wrap your optimizer
optimizer = hvd.DistributedOptimizer(optimizer, named_parameters=model.named_parameters())

# Sync initial weights
hvd.broadcast_parameters(model.state_dict(), root_rank=0)
```

Launch: `horovodrun -np 4 -H server1:2,server2:2 python train.py`

That's roughly it. Horovod intercepts gradient computation, runs an all-reduce across all workers, and every process applies the averaged gradient locally.

Key results from the paper:

- 90% scaling efficiency on 128 GPU servers for ResNet-101.
- Near-linear speedups for training Inception V3, VGG-16, ResNet-101 on multiple GPU nodes.

Paper: "Horovod: fast and easy distributed deep learning in TensorFlow" (Sergeev & Del Balso, 2018)

arXiv: <https://arxiv.org/abs/1802.05799>

Project site: <https://horovod.ai>

GitHub: <https://github.com/horovod/horovod>

Docs: <https://horovod.readthedocs.io>

Using Horovod in your existing project (e.g., InceptionResNet for exoplanet detection):

1. Install: `pip install horovod` (also needs MPI and NCCL).

2. Add the ~10 lines of Horovod boilerplate to your training script.
3. Launch with `horovodrun` specifying host IPs and GPU counts.
4. Each machine must be reachable via SSH with the same environment (ideal for a Docker-based setup).

Difficulty to integrate into existing project: Low–medium. Main friction is setting up MPI across machines and ensuring environment parity (exactly where Docker helps — your Compute4Me containers would ship this pre-configured).

Difficulty to reimplement from scratch: Very high. Years of engineering for gradient bucketing, tensor fusion, overlapping comms with compute, fault detection, etc. Not worth reimplementing.

Relevance to Compute4Me: Horovod can be the training backend inside worker containers. The Compute4Me orchestration layer decides which containers get which data shards, and each container uses Horovod internally to synchronize gradients if multi-GPU training is needed within a single node.

Key System: PyTorch DistributedDataParallel (DDP)

PyTorch DDP is now the canonical built-in way to do data-parallel distributed training with PyTorch. It wraps your `nn.Module` and uses NCCL (or Gloo for CPU) for all-reduce under the hood.

How it works:

1. You launch one process per GPU (using `torchrun` or `torch.multiprocessing.spawn`).
2. Each process creates its own copy of the model.
3. Wrap: `model = torch.nn.parallel.DistributedDataParallel(model, device_ids=[local_rank])`.
4. When `.backward()` is called, DDP registers hooks on each parameter that trigger an all-reduce of gradients across all processes before the optimizer step.

Because all processes apply the same averaged gradient, models stay in sync without any PS.

Key environment variables you set:

- `MASTER_ADDR` — IP of the rank-0 process (the "coordinator")
- `MASTER_PORT` — Port for coordination
- `WORLD_SIZE` — Total number of processes
- `RANK` — This process's global rank (0 to `WORLD_SIZE-1`)
- `LOCAL_RANK` — Rank within the current node

DDP design notes: <https://github.com/pytorch/pytorch/blob/main/docs/source/notes/ddp.rst>

Tutorial: https://github.com/pytorch/tutorials/blob/main/intermediate_source/ddp_tutorial.rst

Difficulty to use: Low–medium. Setting env vars and wrapping the model is straightforward. The trickier part is managing the process launch across multiple machines — which is where Compute4Me's orchestration layer would help enormously.

Difficulty to reimplement: Not worth it.

Relevance to Compute4Me: PyTorch DDP is likely your default training backend inside each worker container. Your master just needs to:

- Know the IPs/ports of all participating containers.
- Set MASTER_ADDR, MASTER_PORT, WORLD_SIZE, and assign each container a RANK.
- Start containers with the right env vars — then DDP handles the rest.

2.3 PS vs All-reduce: Comparison

Dimension	Parameter Server	All-reduce (Ring)
Architecture	Central server(s) + workers	Peer-to-peer among workers
Synchrony	Supports async (Downpour SGD) or sync	Synchronous only
Straggler sensitivity	Low (async mode tolerates slow nodes)	High (all workers must sync each step)
Best for	Heterogeneous, large, or unstable clusters	Homogeneous, fast-interconnect clusters
Communication overhead	$O(N \times \text{model size})$ for naive PS; can be reduced with sharding	$O(2 \times \text{model size})$ per worker regardless of N — bandwidth-optimal
Implementation complexity	Moderate (need PS server code + RPC layer)	Low (just use NCCL/Gloo + DDP)
Compute4Me fit	Better fit for volunteer/heterogeneous nodes	Better fit within homogeneous sub-groups
Example systems	DistBelief, GeePS, PS-Lite, MXNet KVStore	Horovod, PyTorch DDP, DeepSpeed ZeRO

2.4 What Is "Solved" vs Still Challenging

Solved / Commoditized (Do Not Reinvent)

- Gradient synchronization via all-reduce on homogeneous clusters: just use PyTorch DDP + NCCL.
- Multi-framework distributed training via Horovod.
- High-performance collective comms via NCCL.
- Basic PS pattern for async training: well understood and can be implemented with standard tools.

Still Challenging (Relevant Research Areas for Compute4Me)

- **Heterogeneous node handling:** Both PS and all-reduce assume workers are reasonably similar in speed. Highly heterogeneous setups (RTX 3070 vs a CPU-only VM vs an old Tesla K80) still cause problems.
 - Async PS helps but convergence is noisier.
 - Ring all-reduce breaks down when one worker is 10× slower.
- **Dynamic membership:** Workers joining or leaving mid-training breaks both architectures unless you engineer elasticity on top (see Section 4 of the main report).
- **Bandwidth-constrained volunteer networks:** Both architectures assume reasonable network bandwidth. Over commodity internet (10–100 Mbps), synchronizing gradients every step becomes the bottleneck.

2.5 Practical Takeaway for Compute4Me

The classical distributed DL architectures give you two reusable backend options:

1. **Use PyTorch DDP + NCCL inside containers:** When you assign a batch of similar-speed worker containers to one training job, they can coordinate via DDP internally. Your master just sets the env vars and they sort themselves out.
2. **Use an async PS pattern for the global coordination layer:** If nodes are very heterogeneous or unreliable, consider having your master act as a lightweight PS — workers push gradients asynchronously when ready, and the master maintains the canonical model checkpoint. This is closer to DistBelief's Downpour SGD and is more resilient to stragglers.

In either case, you do not need to reimplement gradient communication primitives. The interesting engineering for Compute4Me is everything *above* this layer: capability discovery, task scheduling, elasticity, fault tolerance, and the container orchestration protocol.

Next: Section 3 — Ray and General-Purpose AI Compute Engines